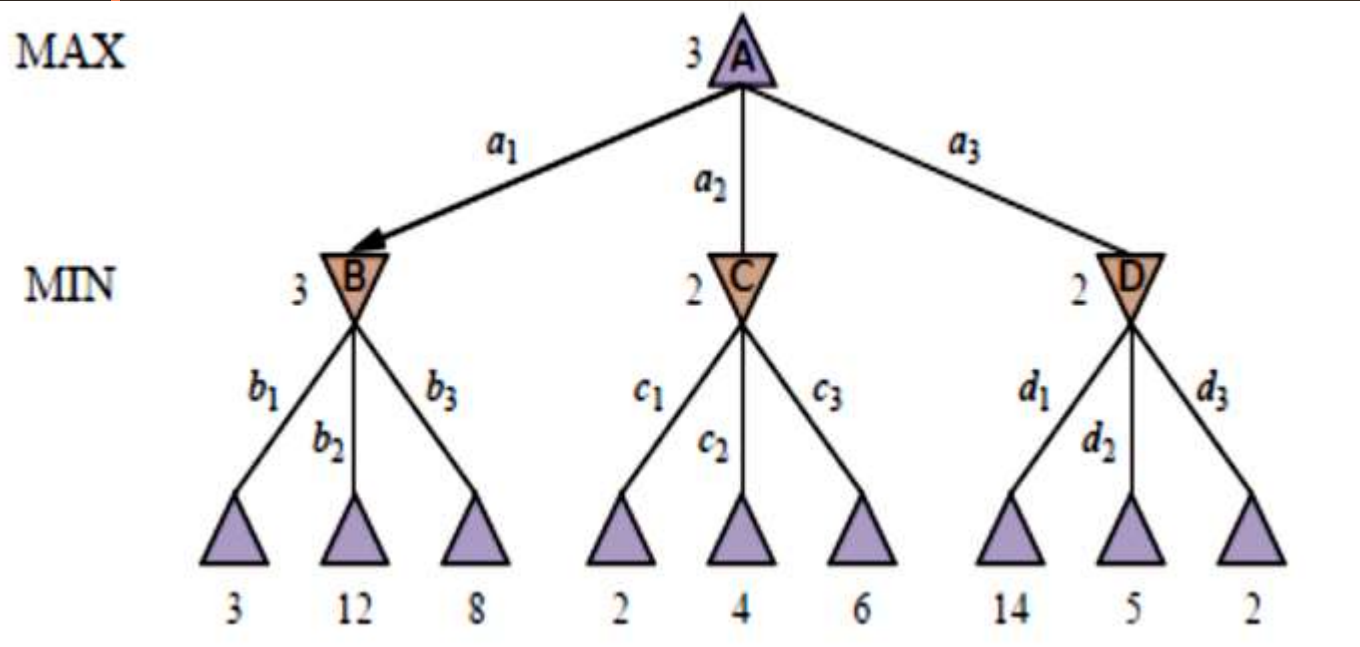


ALPHA-BETA PRUNING

- Modified version of the minimax algorithm → an optimization technique
- Minimax search alg. is exponential over 'd' → this can be cut to half
- Pruning → a technique by which without checking each node of the game tree the correct minimax decision can be computed → computing the correct minimax decision without examining every state by pruning large parts of the tree that make no difference to the outcome → can be applied at any depth of a tree → can prune the tree leaves and also an entire sub-tree.
- Alpha – beta pruning → the technique that uses pruning and 2 parameters (Alpha and Beta) on Minimax alg.

ALPHA-BETA PRUNING

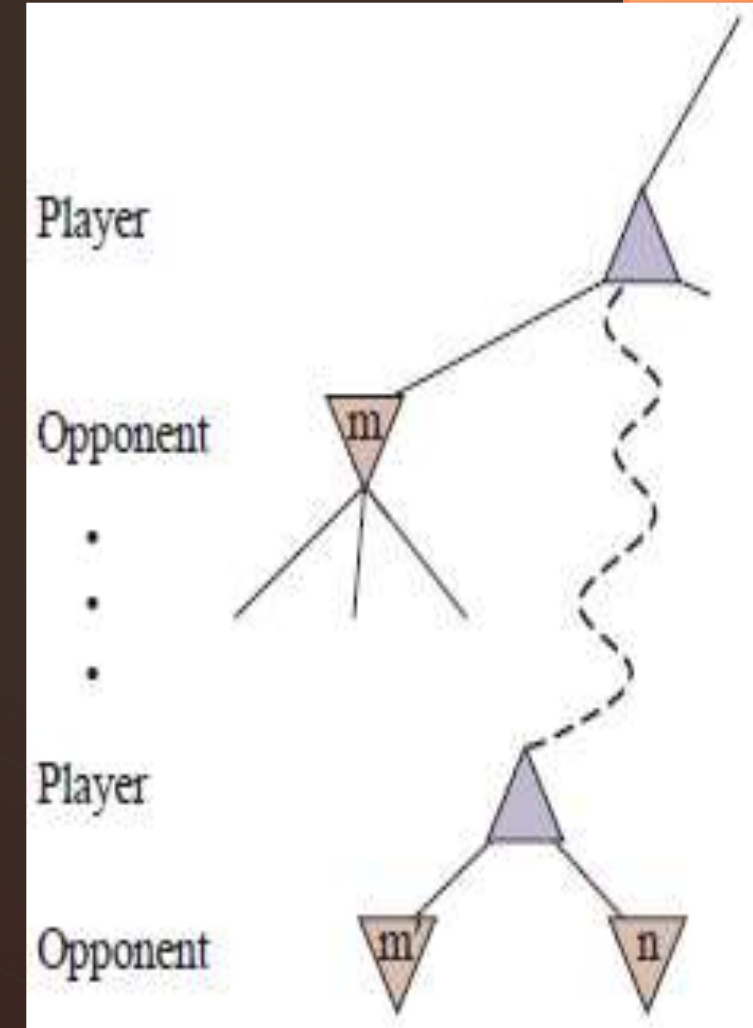


$$\begin{aligned}\text{MINIMAX}(A) &= \max(B, C, D) \\ &= \max(\min(3, 12, 8), \min(2, 4, 6), \min(14, 5, 2)) \\ &= \max(3, 2, 2) = 3\end{aligned}$$

- If actions c_2 and c_3 lead to utility values x & y (instead of 4 & 6):
- $\text{MINIMAX}(A) = \max(B, C, D) = \max(\min(3, 12, 8), \min(2, x, y), \min(14, 5, 2))$
- $= \max(3, \min(2, x, y), 2) = \max(3, z, 2)$ where $z = \min(2, x, y) \leq 2 = 3$

ALPHA-BETA PRUNING

- General principle:
- Consider a node n somewhere in the tree, such that Player has a choice of moving to n .
- If Player has a better choice either at the same level or at any point higher up in the tree, then Player will never move to n .
- So once enough details is found out about n (by examining some of its descendants) to reach this conclusion (moving or not moving to n), it can be pruned.
- Eg. if n is worse than m or m' , n can be pruned.



ALPHA-BETA PRUNING

- 2 parameters used apart from state $s \rightarrow \alpha, \beta$
- α = the value of the best (i.e., highest-value) choice found so far at any choice point along the path for MAX = “at least” = $-\infty$
- β = the value of the best (i.e., lowest-value) choice found so far at any choice point along the path for MIN = “at most” = $+\infty$
- The alg. updates the values of α and β .
- Then prunes the remaining branches at a node as soon as the value of the current node is known to be worse than the current α or β value for MAX or MIN, respectively.
- To prune $\rightarrow$ the algorithm terminates the recursive call $\rightarrow$ occurs when $\alpha \geq \beta$

ALPHA-BETA PRUNING

Condition for Alpha-beta pruning

- Alpha: At any point along the Maximizer path, Alpha is the best option or the highest value we've discovered. The initial value for alpha is $-\infty$.
- Beta: At any point along the Minimizer path, Beta is the best option or the lowest value we've discovered.. The initial value for beta is $+\infty$.
- The condition for Alpha-beta Pruning is that $\alpha \geq \beta$.
- The alpha and beta values of each node must be kept track of. Alpha can only be updated when it's MAX's time, and beta can only be updated when it's MIN's turn.
- MAX will update only alpha values and the MIN player will update only beta values.
- The node values will be passed to upper nodes instead of alpha and beta values during going into the tree's reverse.
- Alpha and Beta values only are passed to child nodes.

ALPHA-BETA PRUNING

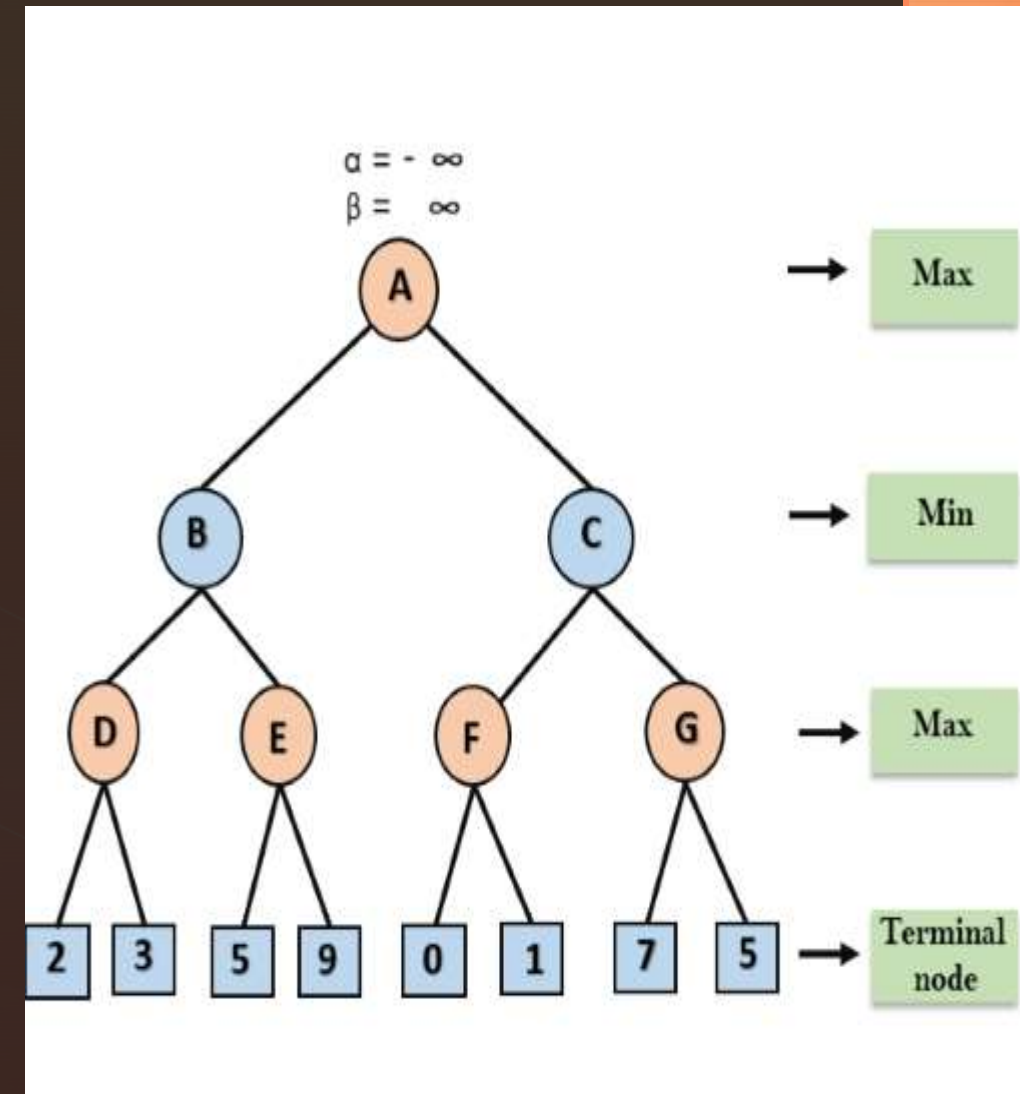
function ALPHA-BETA-SEARCH(*game, state*) **returns** an action
 $\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$
 return *move*

function MAX-VALUE(*game, state, α , β*) **returns** a (*utility, move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state, player*), null
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** *v, move*
 return *v, move*

function MIN-VALUE(*game, state, α , β*) **returns** a (*utility, move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state, player*), null
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** *v, move*
 return *v, move*

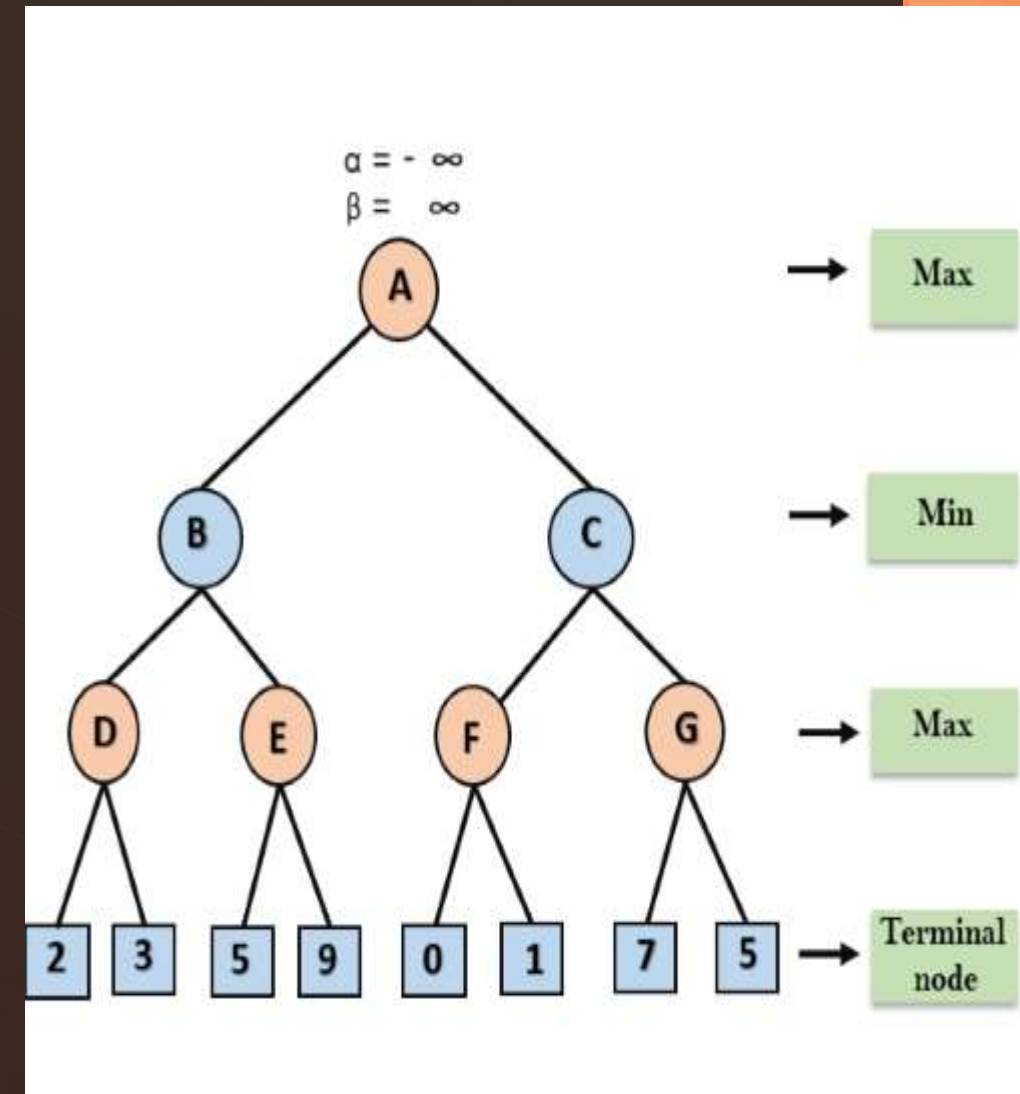
ALPHA-BETA PRUNING

- **Step-1:** At the first step the, Max player will start first move from node A where $\alpha = -\infty$ and $\beta = +\infty$,
- These values of alpha and beta passed down to node B where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



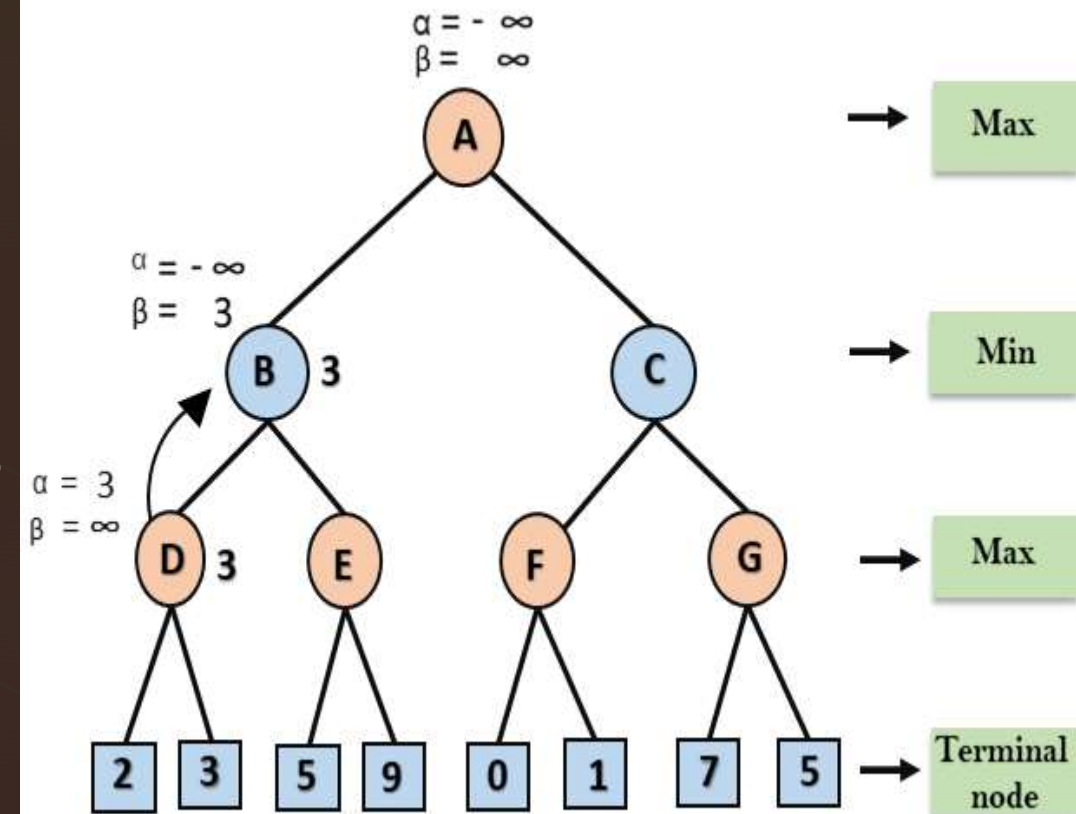
ALPHA-BETA PRUNING

- **Step-2:** At Node D, the value of α will be calculated as its Max's turn. The value of $\alpha = -\infty$ is compared with 2 $\rightarrow \max(-\infty, 2) = 2$ and then 2 is compared with 3, and at node D, $\alpha = \max(2, 3) = 3$



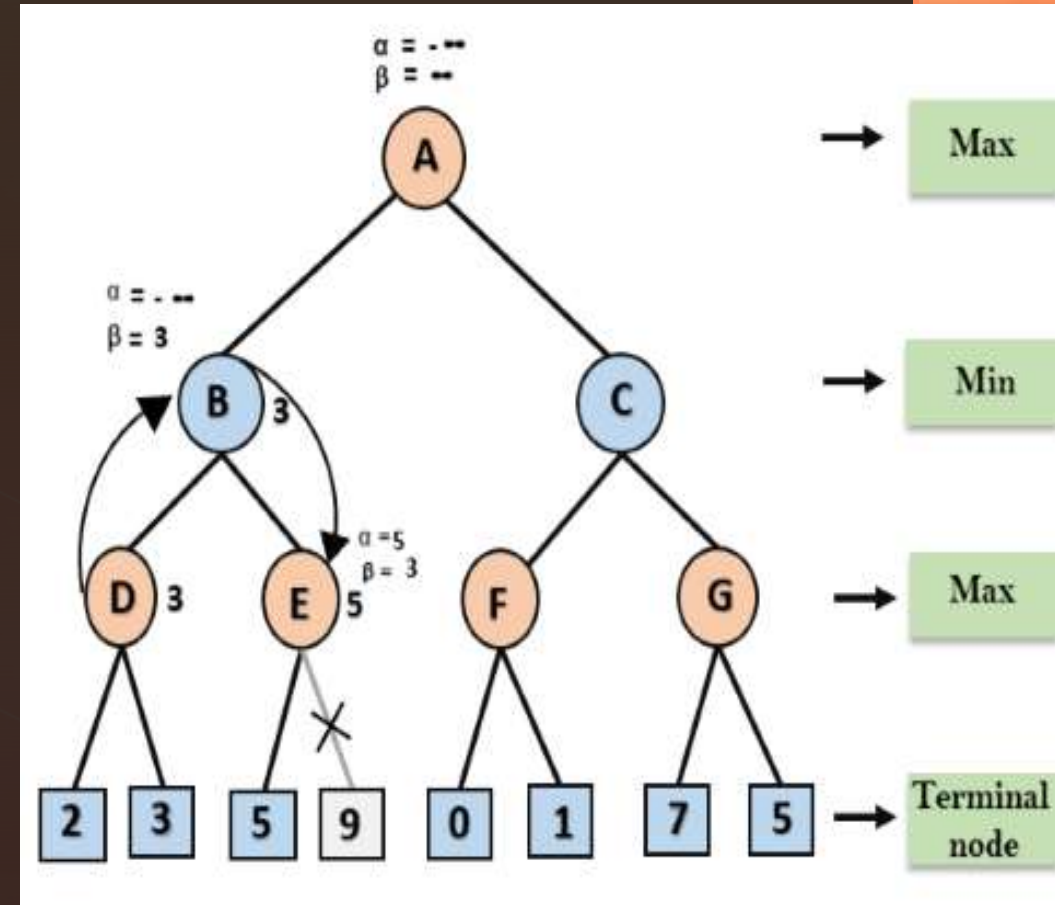
ALPHA-BETA PRUNING

- **Step-3:** Now algorithm backtracks to node B, where the value of β will change as this is Min's turn,
- Now $\beta = +\infty$, will be compared with the available subsequent nodes value, i.e. $\min(\infty, 3) = 3$, hence at node B now $\alpha = -\infty$, and $\beta = 3$.
- In the next step, algorithm traverse the next successor of Node B which is node E, and the values of $\alpha = -\infty$, and $\beta = 3$ will also be passed.



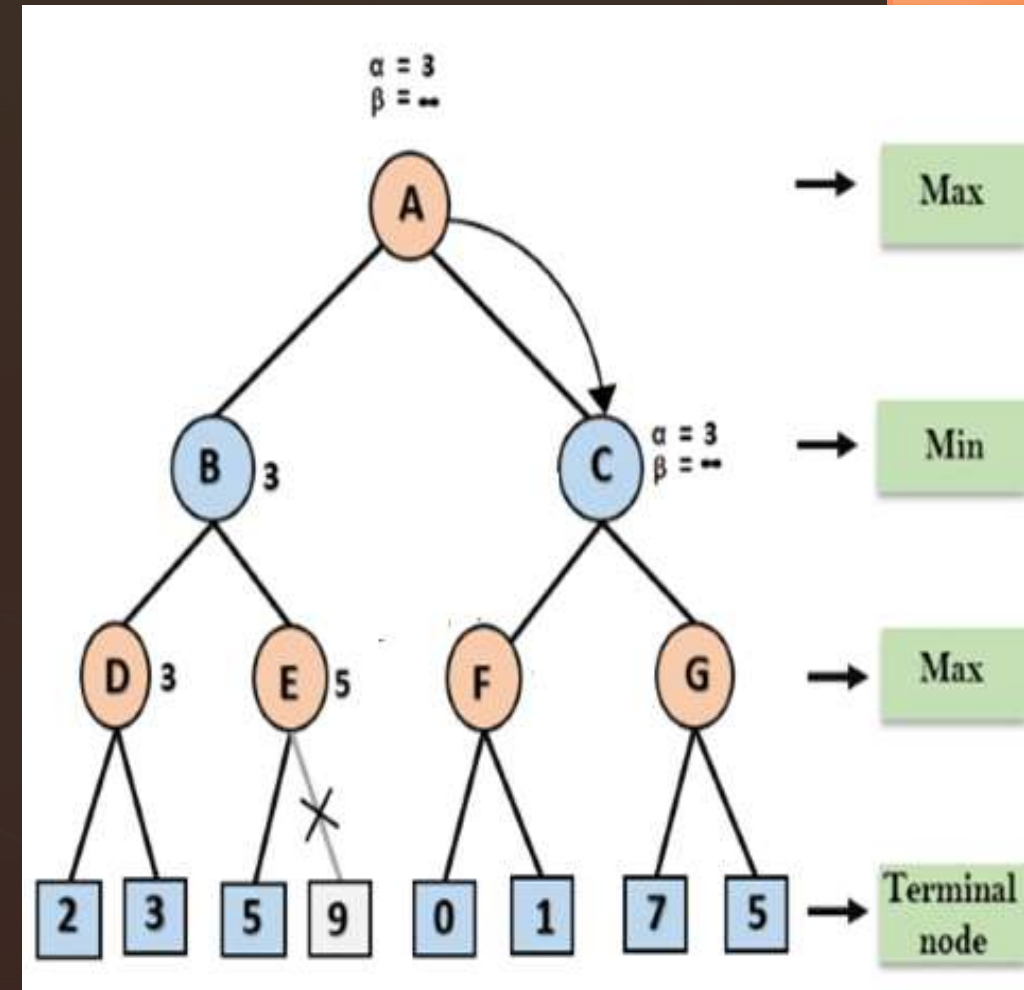
ALPHA-BETA PRUNING

- **Step-4:** At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha \geq \beta$, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



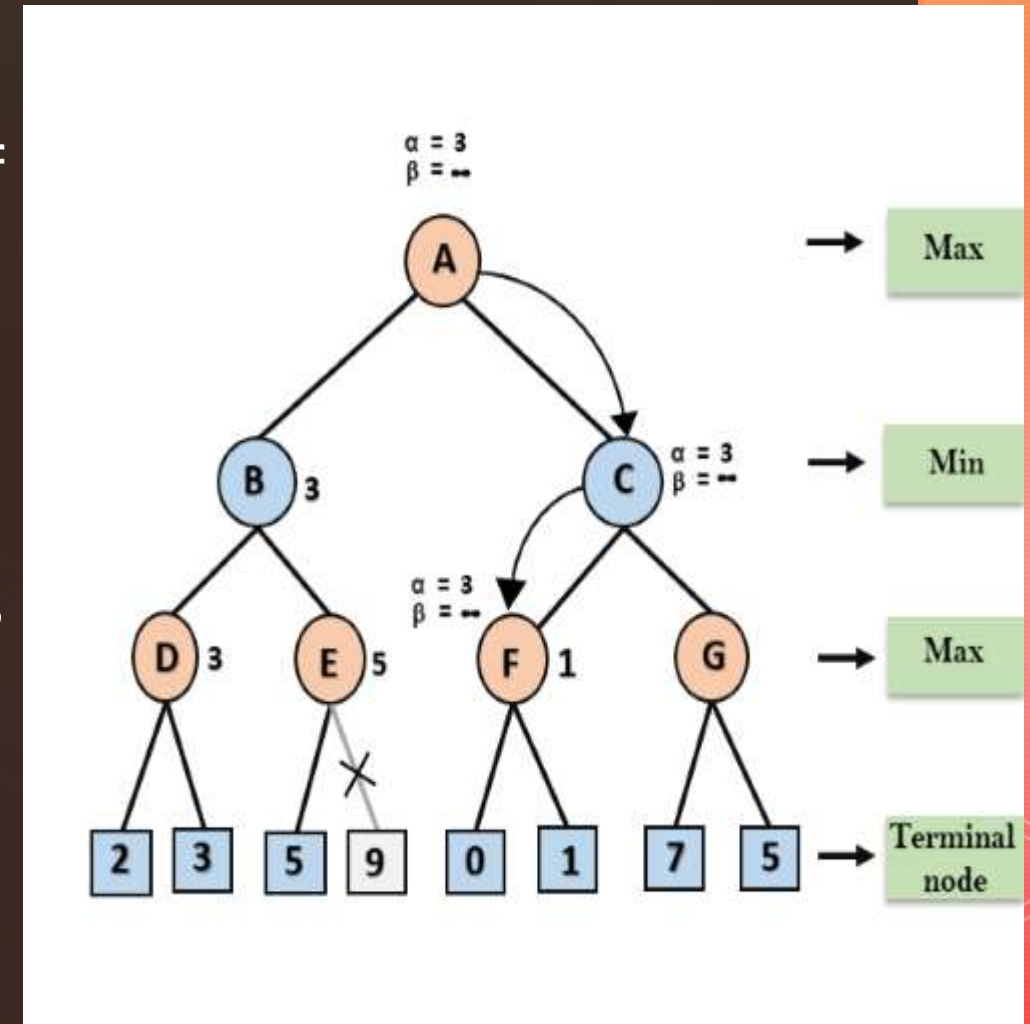
ALPHA-BETA PRUNING

- **Step-5:** At next step, algorithm again backtracks the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as $\max(-\infty, 3) = 3$, and $\beta = +\infty$, these two values now pass to right successor of A which is Node C.
- At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.



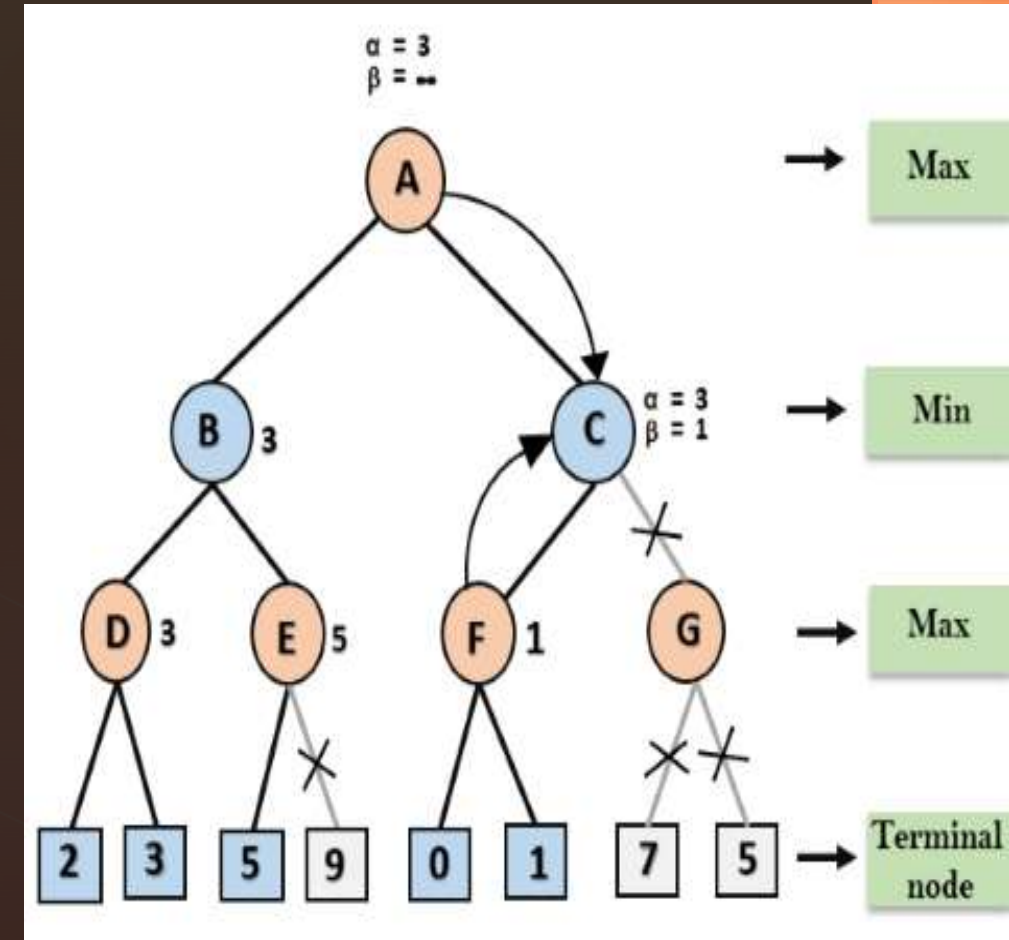
ALPHA-BETA PRUNING

- Step-6:** At node F, again the value of α will be compared with left child which is 0, and $\max(3,0)=3$, and then compared with right child which is 1, and $\max(3,1)=3$ still α remains 3, but the node value of F will become 1.



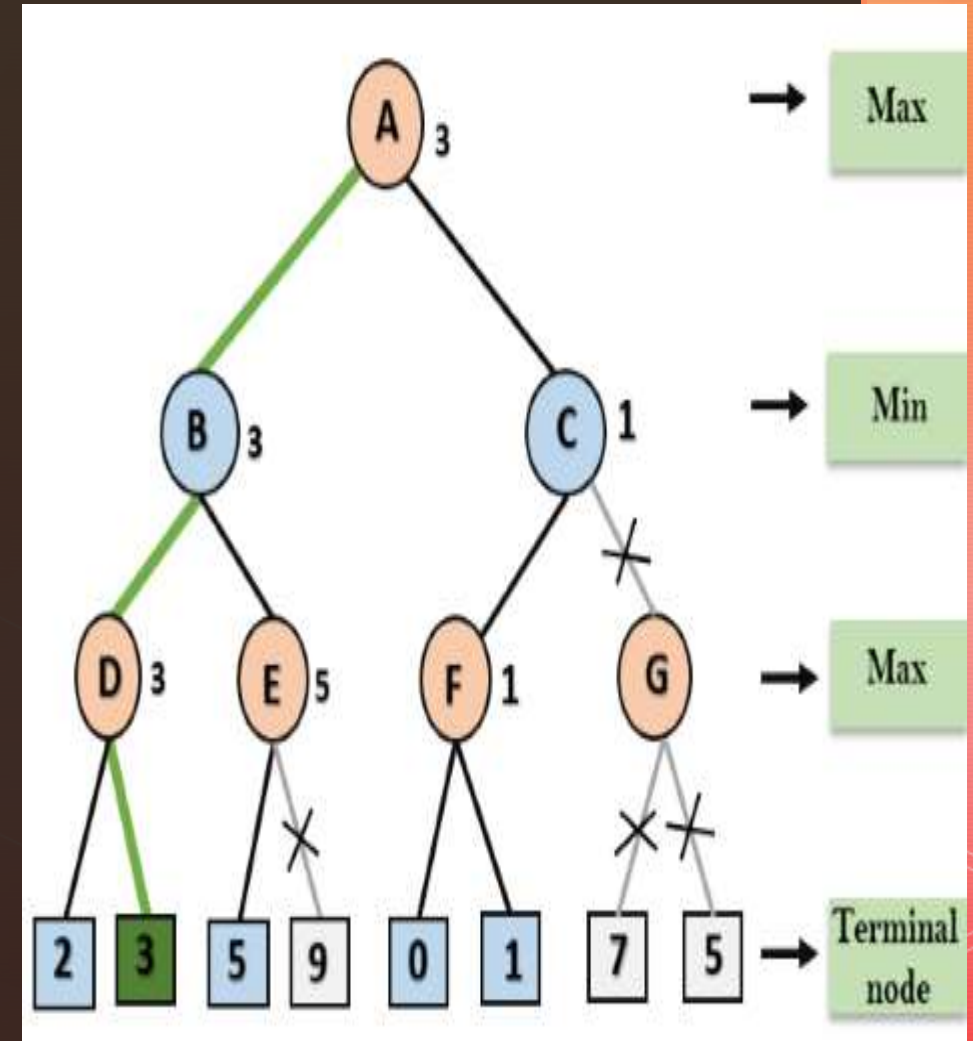
ALPHA-BETA PRUNING

- **Step-7:** Node F returns the node value 1 to node C, at C $\alpha=3$ and $\beta=+\infty$, here the value of β will be changed, by compared with 1 so $\min(\infty, 1) = 1$.
- Now at C, $\alpha=3$ and $\beta=1$, and again it satisfies the condition $\alpha \geq \beta$, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.
- Value at node C becomes 1, as it is Min's turn



ALPHA-BETA PRUNING

- **Step-8:** Node C now returns the value of 1 to A here the best value for A is $\max(3, 1) = 3$. The path in green is the final game tree.
- It shows the nodes which are computed and nodes which were never computed.
- Hence the optimal value for the maximizer is 3 for this example.



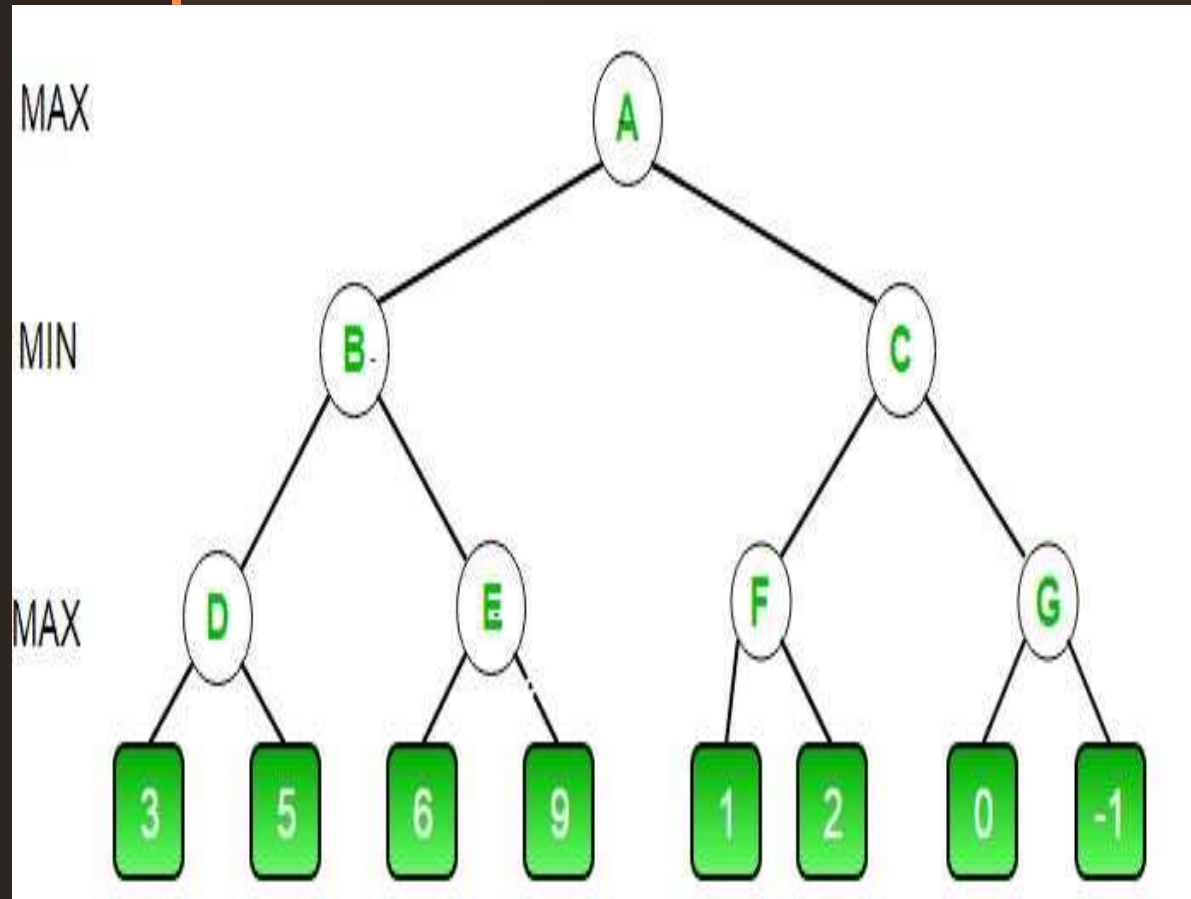
ALPHA-BETA PRUNING

NODE	TURN	STEP	VALUE AT NODE	α	β	$\alpha \geq \beta$
At A	Max's			$-\infty$	$+\infty$	False
At B	Min's	Values passed from A		$-\infty$	$+\infty$	False
At D	Max's	Values passed from B		$-\infty$	$+\infty$	False
At D		$\alpha = \text{Max}(\alpha, 2) = \text{Max}(-\infty, 2) = 2$	2	2	$+\infty$	False
At D		$\alpha = \text{Max}(\alpha, 3) = \text{Max}(2, 3) = 3$	3	3	$+\infty$	False
At D			$\text{Max}(2, 3) = 3$			
At B	Min's	Gets 3 from D	3	$-\infty$	3	False
At E	Max's	Values passed from B		$-\infty$	3	False
At E		$\alpha = \text{Max}(\alpha, 5) = \text{Max}(-\infty, 5) = 5$	5	5	3	True
At E		PRUNE THE NEXT SUCCESSOR	5			

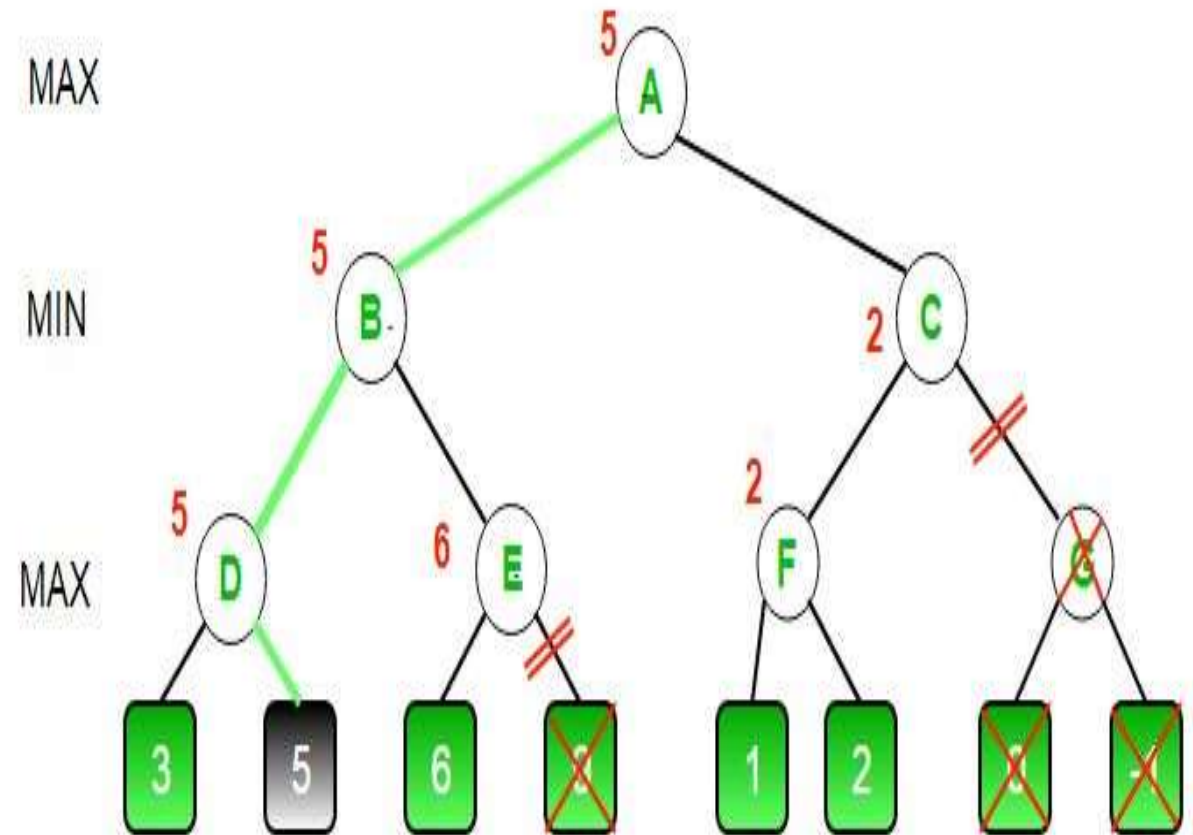
ALPHA-BETA PRUNING						
Node	Turn	Step	Value at Node	α	β	$\alpha \geq \beta$
At B	Min's	Gets 5 from E	$\text{Min}(3, 5) = 3$			
At A	Max's	Gets 3 from B	3	3	$+\infty$	False
At C	Min's	Values passed from A		3	$+\infty$	False
At F	Max's	Values passed from C		3	$+\infty$	False
At F		$\alpha = \text{Max}(\alpha, 0) = \text{Max}(3, 0) = 3$	0	3	$+\infty$	False
At F		$\alpha = \text{Max}(\alpha, 1) = \text{Max}(3, 1) = 3$	1	3	$+\infty$	False
At F			$\text{Max}(0, 1) = 1$			
At C	Min's	Gets 1 from F		3	1	True
At C	Min's	PRUNE THE NEXT SUCCESSOR	1	3	1	True
At A	Max's	Gets 1 from C	$\text{Max}(3, 1) = 3$			

ALPHA-BETA PRUNING

Example 2:



Solution: Search path



ALPHA-BETA PRUNING

- The initial call starts from **A**. The value of alpha here is **-INFINITY** and the value of beta is **+INFINITY**. These values are passed down to subsequent nodes in the tree. At **A** the maximizer must choose max of **B** and **C**, so **A** calls **B** first
- At **B** it the minimizer must choose min of **D** and **E** and hence calls **D** first.
- At **D**, it looks at its left child which is a leaf node. This node returns a value of 3. Now the value of alpha at **D** is $\max(-\text{INF}, 3)$ which is 3.
- To decide whether its worth looking at its right node or not, it checks the condition $\beta \leq \alpha$. This is false since $\beta = +\text{INF}$ and $\alpha = 3$. So it continues the search.
- **D** now looks at its right child which returns a value of 5. At **D**, $\alpha = \max(3, 5)$ which is 5. Now the value of node **D** is 5

ALPHA-BETA PRUNING

- **D** returns a value of 5 to **B**. At **B**, $\beta = \min(+\infty, 5)$ which is 5. The minimizer is now guaranteed a value of 5 or lesser. **B** now calls **E** to see if he can get a lower value than 5.
- At **E** the values of alpha and beta is not $-\infty$ and $+\infty$ but instead $-\infty$ and 5 respectively, because the value of beta was changed at **B** and that is what **B** passed down to **E**
- Now **E** looks at its left child which is 6. At **E**, $\alpha = \max(-\infty, 6)$ which is 6. Here the condition becomes true. beta is 5 and alpha is 6. So $\beta \leq \alpha$ is true. Hence it breaks and **E** returns 6 to **B**

ALPHA-BETA PRUNING

- Note how it did not matter what the value of **E**'s right child is. It could have been $+\text{INF}$ or $-\text{INF}$, it still wouldn't matter, We never even had to look at it because the minimizer was guaranteed a value of 5 or lesser. So as soon as the maximizer saw the 6 he knew the minimizer would never come this way because he can get a 5 on the left side of **B**. This way we didn't have to look at that 9 and hence saved computation time.
- **E** returns a value of 6 to **B**. At **B**, $\text{beta} = \min(5, 6)$ which is 5. The value of node **B** is also 5
- The 9 is crossed out because it was never computed.
- **B** returns 5 to **A**. At **A**, $\text{alpha} = \max(-\text{INF}, 5)$ which is 5. Now the maximizer is guaranteed a value of 5 or greater. **A** now calls **C** to see if it can get a higher value than 5.
- At **C**, $\text{alpha} = 5$ and $\text{beta} = +\text{INF}$. **C** calls **F**
- At **F**, $\text{alpha} = 5$ and $\text{beta} = +\text{INF}$. **F** looks at its left child which is a 1. $\text{alpha} = \max(5, 1)$ which is still 5.

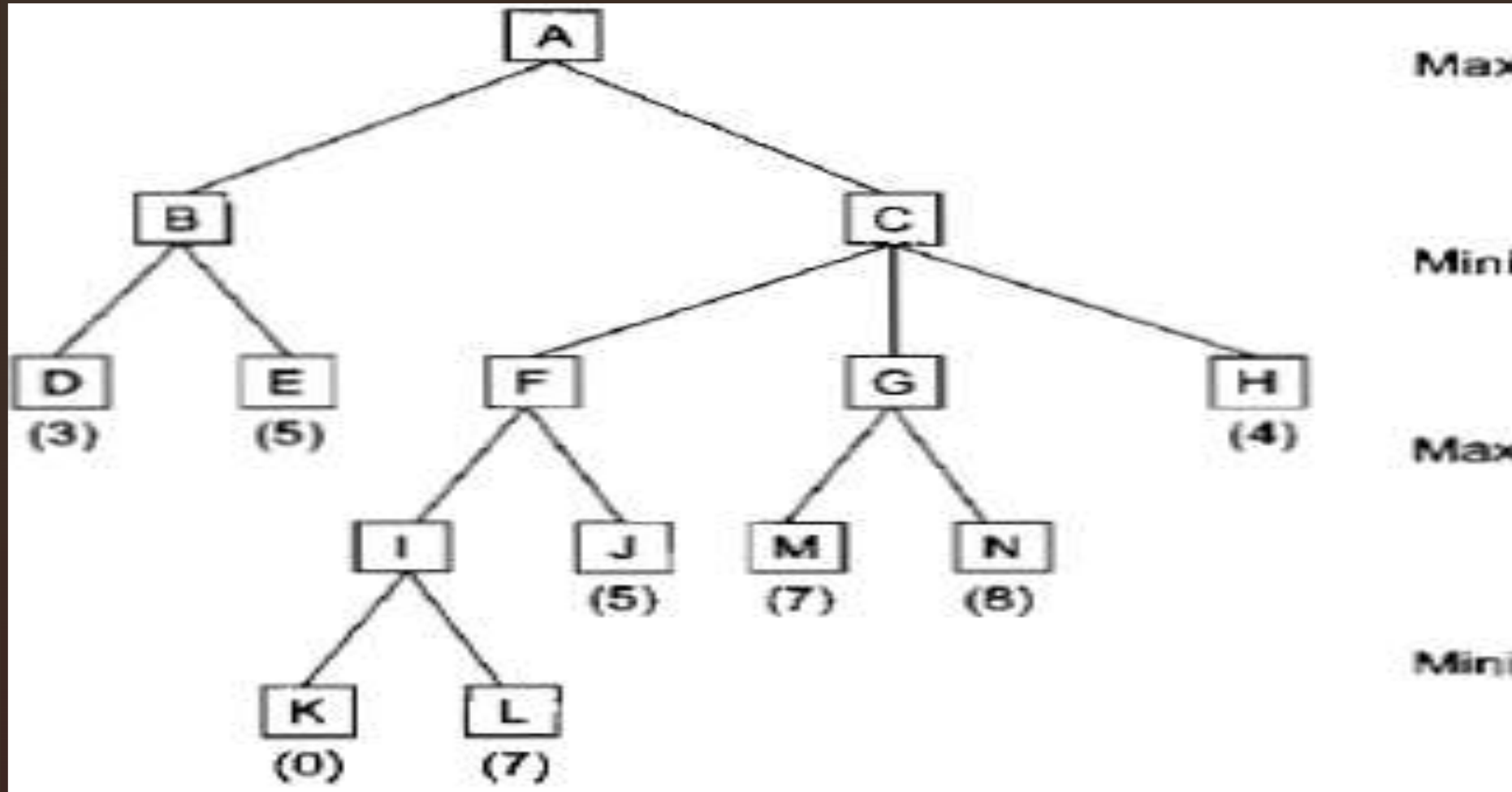
ALPHA-BETA PRUNING

- **F** looks at its right child which is a 2. Hence the best value of this node is 2. Alpha still remains 5
- **F** returns a value of 2 to **C**. At **C**, $\text{beta} = \min(+\text{INF}, 2)$. The condition $\text{beta} \leq \text{alpha}$ becomes true as $\text{beta} = 2$ and $\text{alpha} = 5$. So it breaks and it does not even have to compute the entire sub-tree of **G**.
- The intuition behind this break-off is that, at **C** the minimizer was guaranteed a value of 2 or lesser. But the maximizer was already guaranteed a value of 5 if he choose **B**. So why would the maximizer ever choose **C** and get a value less than 2 ? Again you can see that it did not matter what those last 2 values were. We also saved a lot of computation by skipping a whole sub-tree.
- **C** now returns a value of 2 to **A**. Therefore the best value at **A** is $\max(5, 2)$ which is a 5.
- Hence the optimal value that the maximizer can get is 5

ALPHA-BETA PRUNING

- Example 3

- :



ALPHA-BETA PRUNING

- The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined.
- 2 types of move order:
- **Worst ordering:** when no leaves are pruned in the tree.
- **Ideal ordering:** when lots of pruning happens in the tree, and best moves occur at the left side of the tree.